Lab 13 — Deployment Experiment Report

Compose override · local NPM reverse proxy · CORS · Ubuntu VM

Dr. Pisit Praiwattana, your mentor in 2026

Student ID / รหัสนักศึกษา	6788117
Name / ชื่อ	Pasit Sugrivaka
Laptop OS, VM OS and Docker versions	Laptop: Windows 11 (Docker Desktop 29.8.0, Docker Engine 29.8.0) VM: Ubuntu 26.04.1 LTS on WSL2, Docker Engine 29.1.3, Docker Compose 2.40.3
Repository URL	https://github.com/Sor117/itcs258-lab13

All four experiments use the same gacha-workshop project and the same STUDENT_ID=6788117. The laptop is Windows 11 with Docker Desktop; the Ubuntu server is a WSL2 Ubuntu 26.04 instance with its own Docker Engine, SSH access and storage.

01 — Compose override + direct port / Compose override และพอร์ตตรง

Base + local overlay, started with every -f listed explicitly:

```
cd gacha-workshop
docker compose --env-file .env -f compose.yml -f compose.local.yml config --quiet
docker compose --env-file .env -f compose.yml -f compose.local.yml up -d --build --wait
docker compose --env-file .env -f compose.yml -f compose.local.yml ps
```

config --quiet printed nothing and exited 0. The merged config shows the port that the local overlay adds:

```
services:
  app:
    ports:
      - mode: ingress
        host_ip: 127.0.0.1
        target: 3000
        published: "8085"
        protocol: tcp
```

ps shows the app published on loopback only and PostgreSQL with no host port:

```
NAME                 SERVICE   STATUS                    PORTS
gacha-6788117-app-1   app       Up (healthy)              127.0.0.1:8085->3000/tcp
gacha-6788117-db-1    db        Up (healthy)              5432/tcp
```

In http://localhost:8085/test with Student ID 6788117 I ran Health → Wallet → Recharge 1000 → Exchange 1000 → Draw once → Saved history. Results: 200, 200, 201, 201, 201 (Copper Golem, COMMON) and 200. The saved history came back with the draw row and the two coin transactions.

Merge table from my own config runs:

Command (shortened)	app ports I saw	restart I saw
docker compose config	127.0.0.1:8085 → 3000	–
docker compose -f compose.yml config	none	–
... -f compose.yml -f compose.production.yml -f compose.proxy.yml config	none (npm: 80, 443, 127.0.0.1:81)	unless-stopped
... -f compose.yml -f compose.local.yml -f compose.production.yml config	127.0.0.1:8085 → 3000 still there	unless-stopped

Original local draw: id 1 · Copper Golem (COMMON) · 2026-11-11T04:09:56.243Z (11:09:56 GMT+7). The same row read from the container:

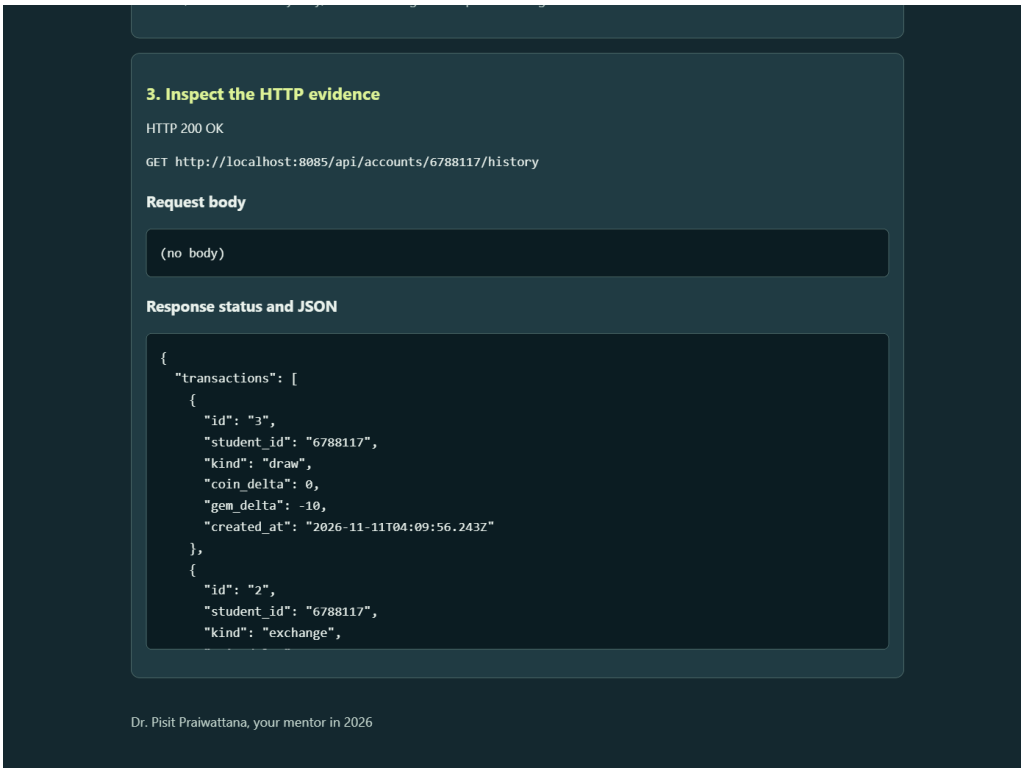
```
docker compose --env-file .env -f compose.yml -f compose.local.yml exec -T db \
  psql -U gacha -d gacha -c "SELECT id, result_name, created_at FROM draws ORDER BY id;"

 id | result_name  |          created_at
----+--------------+-------------------------------
  1 | Copper Golem | 2026-11-11 04:09:56.243929+00
```

Why is compose.override.yml not used when you pass -f? -f replaces the automatic file list. compose.override.yml is only loaded when no -f is given, so an explicit run uses exactly the files I list: base, one environment overlay, then optional helpers.

Why does local + production still publish 8085? ports is a list and Compose adds lists together instead of replacing them. The local overlay still contributes 8085 and the production overlay adds no ports, so 8085 stays published. Only one environment overlay should be used.

Evidence and explanation: The merge is easy to prove: the same yaml file shows 8085 under local and no ports under production. The pitfall is that adding production after local cannot remove the earlier port. config prints real .env values, so I only copied short, sanitized snippets into this report.



Saved history on the direct port: HTTP 200, GET /api/accounts/6788117/history, with the draw transaction at the same time as draw id 1.

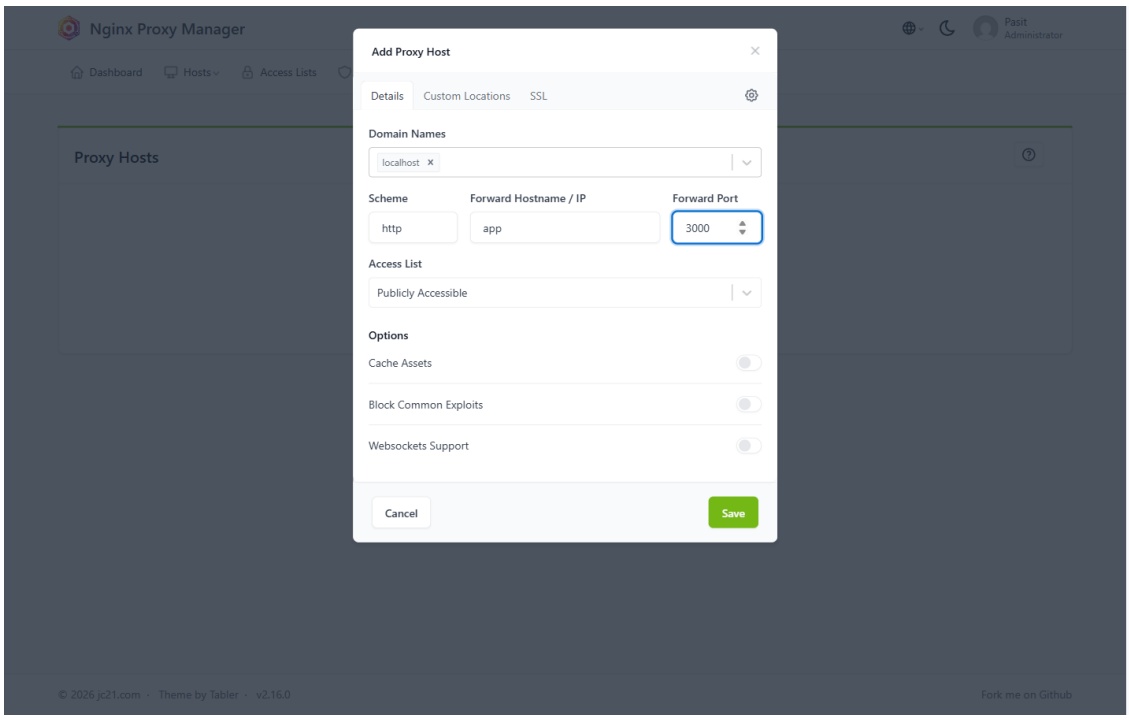
02 — Reverse proxy with local NPM / Reverse proxy ด้วย NPM ในเครื่อง

Production + proxy, with the same project and volume:

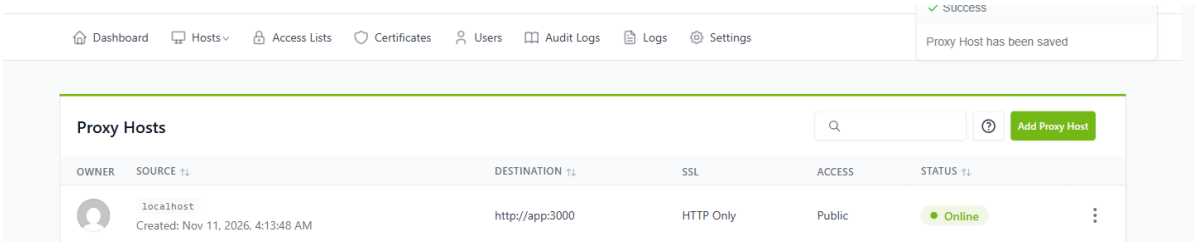
```
docker compose --env-file .env -f compose.yml -f compose.local.yml down
docker compose --env-file .env -f compose.yml -f compose.production.yml \
  -f compose.proxy.yml up -d --build --wait
docker compose --env-file .env -f compose.yml -f compose.production.yml \
  -f compose.proxy.yml ps

NAME                  SERVICE   STATUS       PORTS
gacha-6788117-app-1   app       Up healthy   3000/tcp
gacha-6788117-db-1    db        Up healthy   5432/tcp
gacha-6788117-npm-1   npm       Up           0.0.0.0:80->80, 443->443, 127.0.0.1:81->81
```

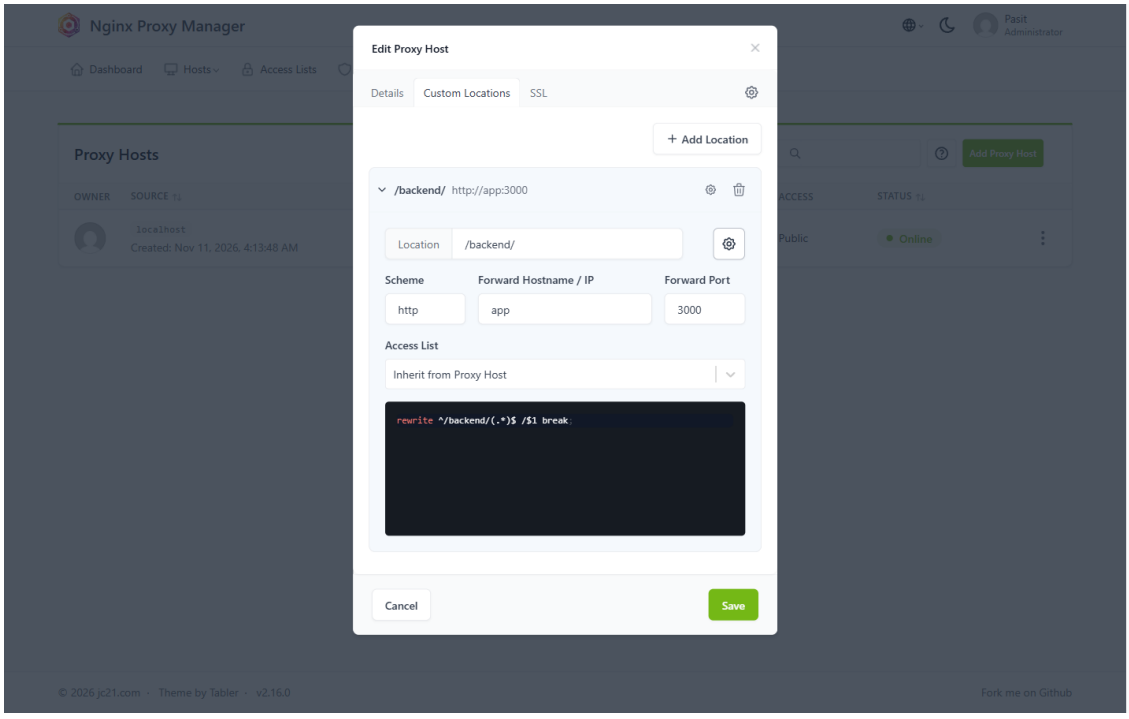
I created the NPM admin at http://127.0.0.1:81 and added the proxy host in the lab order: domain localhost, scheme http, forward app:3000. The four screenshots below are from that NPM instance.



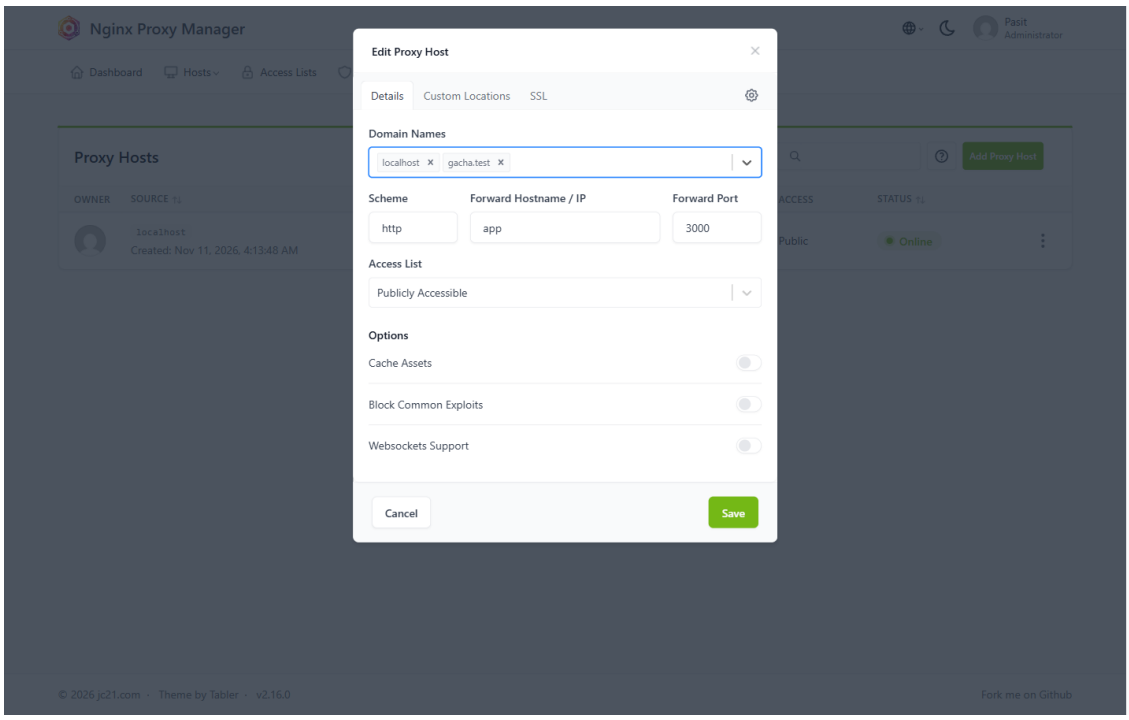
Step 1 Details tab: localhost → http, app, 3000, Publicly Accessible.



Proxy Hosts list: localhost → http://app:3000, HTTP Only, Public, Online.



Step 2 Custom Location /backend/ with the gear editor and the rewrite line.

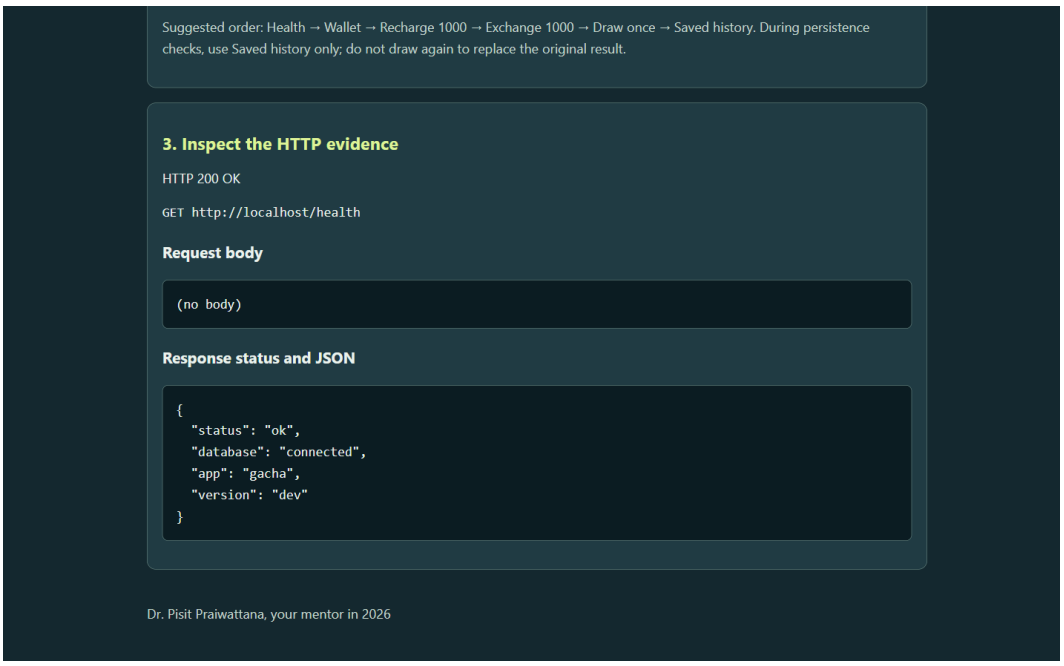


Step 3 Domain Names with both chips: localhost and gacha.test.

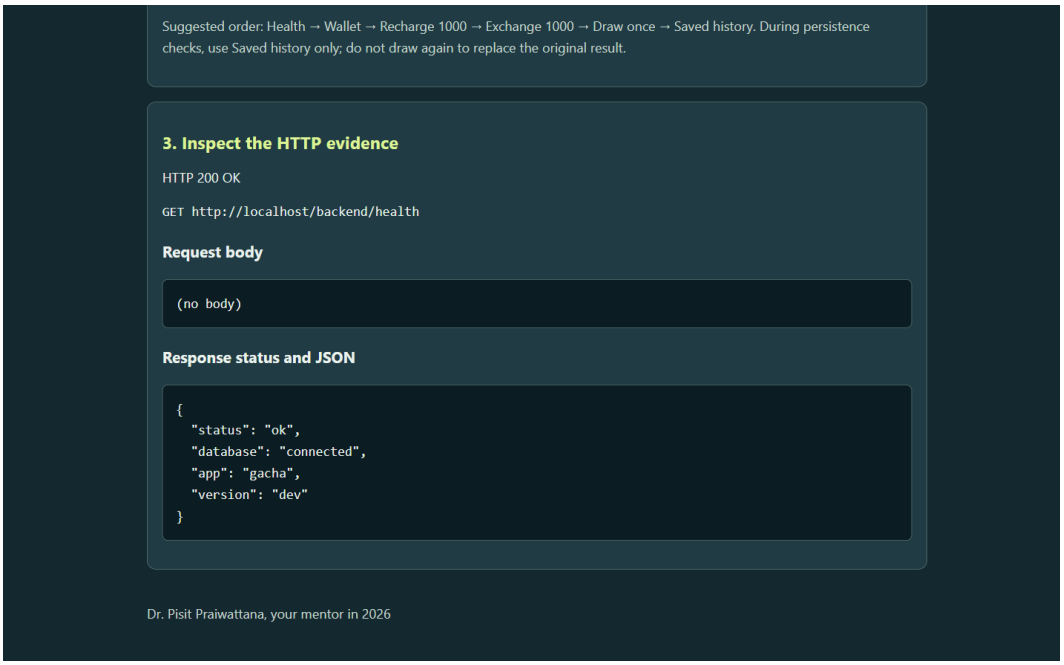
Step test table — what I observed:

Step	URL	Observed status / result
0	http://localhost/ (no proxy host yet)	200 NPM default welcome page
1	http://localhost/health	200 {"status":"ok","database":"connected","app":"gacha"}
1	http://localhost/test → Health	200; GET http://localhost/health
2	http://localhost/backend/test → Health	200; GET http://localhost/backend/health
2	http://localhost/backend/api/catalog	200 JSON
2	http://localhost/backend	301 → Location: http://localhost/backend/
2	http://localhost/backend/	200 app root JSON
2	http://localhost/backendX	404 Route not found
3	http://gacha.test/backend/test before adding gacha.test	404, Server: openresty (no proxy host matched the Host header)
3	http://gacha.test/backend/test after adding it	200; GET http://gacha.test/backend/health
3	curl.exe -v http://gacha.test/backend/health	Trying 127.0.0.1:80… 200 OK

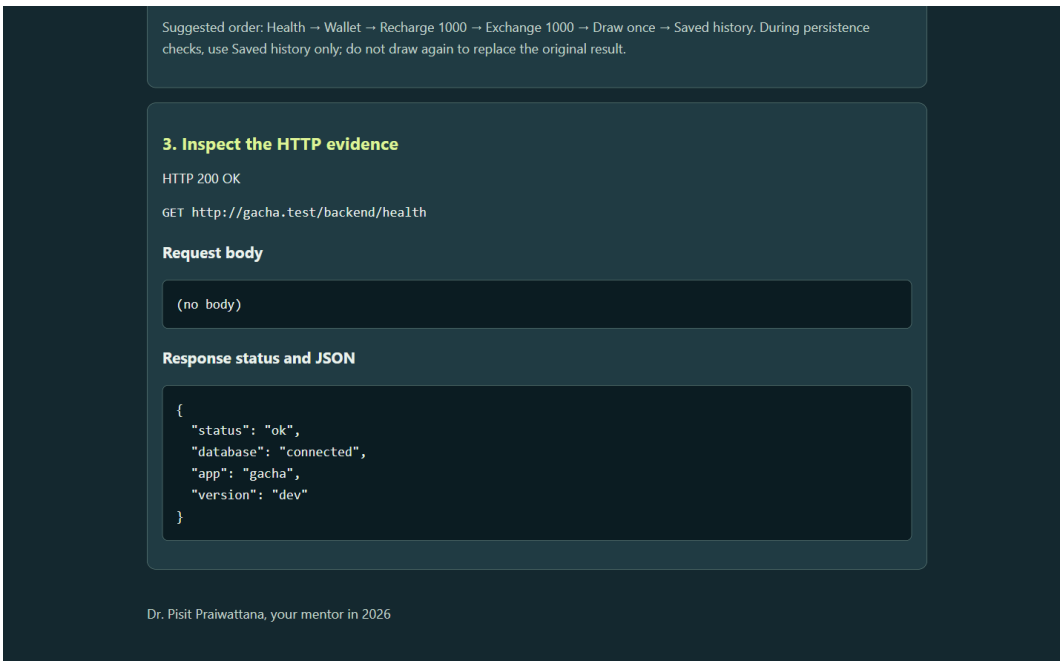
Browser results for the three required URLs:



http://localhost/test → Health: GET http://localhost/health, HTTP 200.



http://localhost/backend/test → Health: GET http://localhost/backend/health, HTTP 200.



http://gacha.test/backend/test → Health: GET http://gacha.test/backend/health, HTTP 200.

/backend/ versus /backend: Nginx matches the location /backend/ only, so /backend (no trailing slash) does not match it and NPM answers 301 to /backend/; the app then serves the root JSON. /backendX is not matched either, so location / sends it unchanged to the app, which answers 404 Route not found. If the location were /backend (no slash), /backend itself would be sent to the app as 404 and /backendX would be captured by that location.

Request path: Laptop browser → NPM :80 (Host: gacha.test, path: /backend/test) → app:3000 (path: /test) → db:5432

Item	Direct local (01)	Local proxy (02)
Browser URL	http://localhost:8085/test	http://gacha.test/backend/test
Host entry port	8085 (published by compose.local.yml)	80 (published by NPM; the app has no host port)
App service/internal port	3000	3000
Database volume name	gacha-6788117_pgdata	gacha-6788117_pgdata
Original draw ID + created_at	1 / 2026-11-11T04:09:56.243Z	1 / 2026-11-11T04:09:56.243Z

Local versus server: On the laptop gacha.test is 127.0.0.1 inside the Windows hosts file. On the Ubuntu VM the laptop hosts file points to the VM address and the VM needs no hosts change; NPM admin is reached through an SSH tunnel because port 81 stays on loopback. A public VPS uses a real DNS A/AAAA record instead of hosts, opens only 80/443 and SSH, and gets HTTPS from NPM's SSL tab with a real domain.

Why localhost inside NPM is never my app (use app:3000): Inside the npm container localhost is the container itself, not the laptop. Port 3000 there is NPM's own admin API, so it returns NPM's 404 instead of the app, and the hostname localhost cannot be resolved by the nginx resolver, which gives 502. The app is only reachable by its Compose service name app on the shared project network.

Why STATUS Online does not prove the upstream works: Online only means nginx accepted and loaded the generated configuration. It does not test the upstream: a wrong forward host still shows Online and fails with 502 at request time.

I tried the optional experiment: I changed Forward Hostname / IP to localhost, saved, and opened /health. NPM still showed Online but returned 502, and its error log said localhost could not be resolved. The custom location (/backend/ → app:3000) kept working, which proved only the root location was wrong. I set the value back to app and /health returned 200 again.

```
2026/11/11 04:18:25 [error] 353#353: *195 localhost could not be resolved (3: Host not found),
client: 172.20.0.1, server: gacha.test, request: "GET /health HTTP/1.1", host: "localhost"
```

The 01 draw read through the proxy, without drawing again:

```
curl.exe -s http://localhost/backend/api/accounts/6788117/history
{"transactions":[...],"draws":[{"id":"1","student_id":"6788117","result_id":"copper-golem",
"result_name":"Copper Golem","rarity":"COMMON","transaction_id":"3",
"created_at":"2026-11-11T04:09:56.243Z"}]}
```

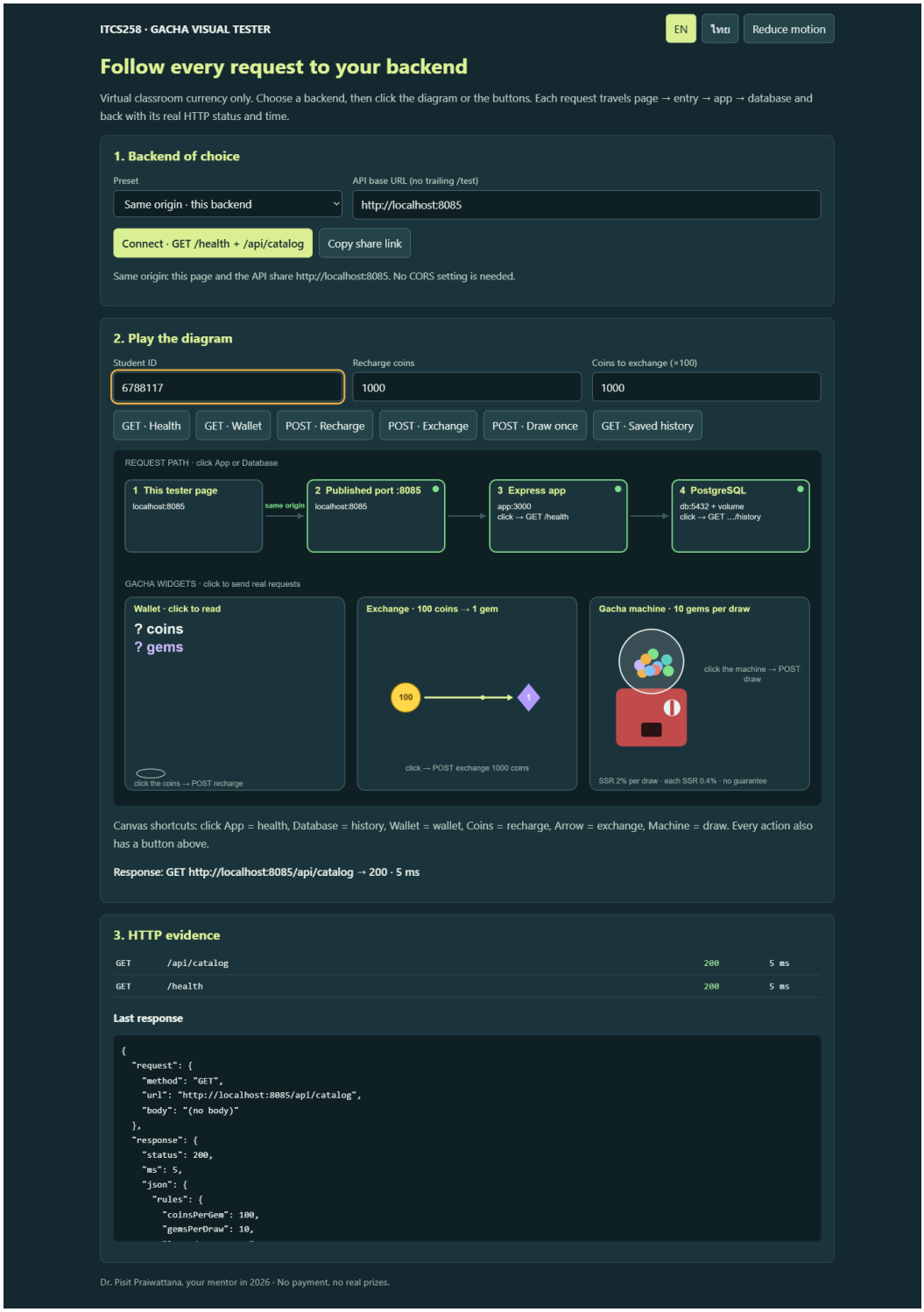
03 — CORS experiment with Docker Compose / ทดลอง CORS ด้วย Docker Compose

Local + CORS overlay (app 8085, db internal, tester 5500):

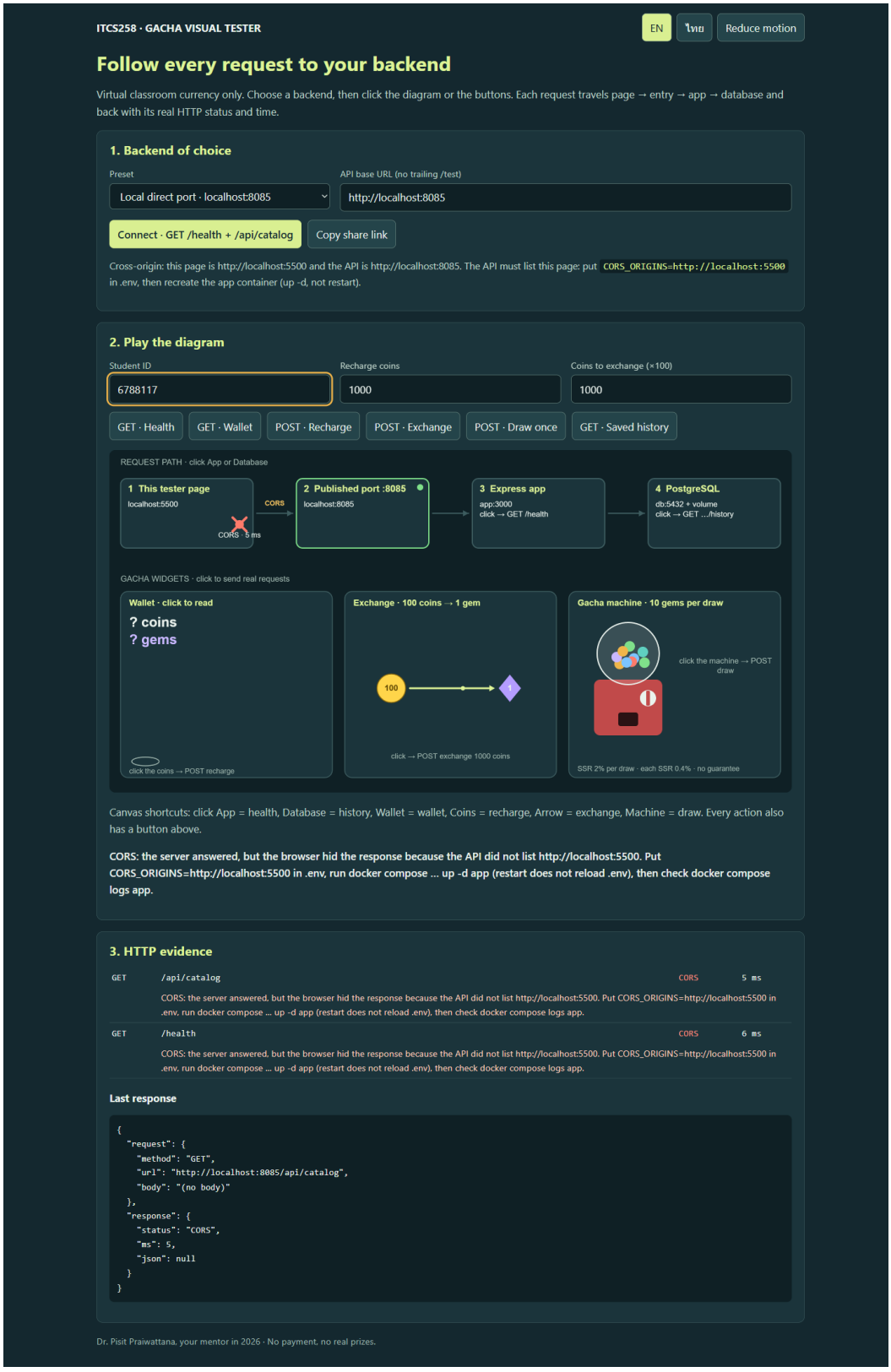
```
docker compose --env-file .env -f compose.yml -f compose.local.yml -f compose.cors.yml up -d --build --
wait
docker compose --env-file .env -f compose.yml -f compose.local.yml -f compose.cors.yml ps

NAME                     SERVICE   STATUS       PORTS
gacha-6788117-app-1      app       Up (healthy) 127.0.0.1:8085->3000/tcp
gacha-6788117-db-1       db        Up (healthy) 5432/tcp
gacha-6788117-tester-1   tester    Up (healthy) 127.0.0.1:5500->5500/tcp
```

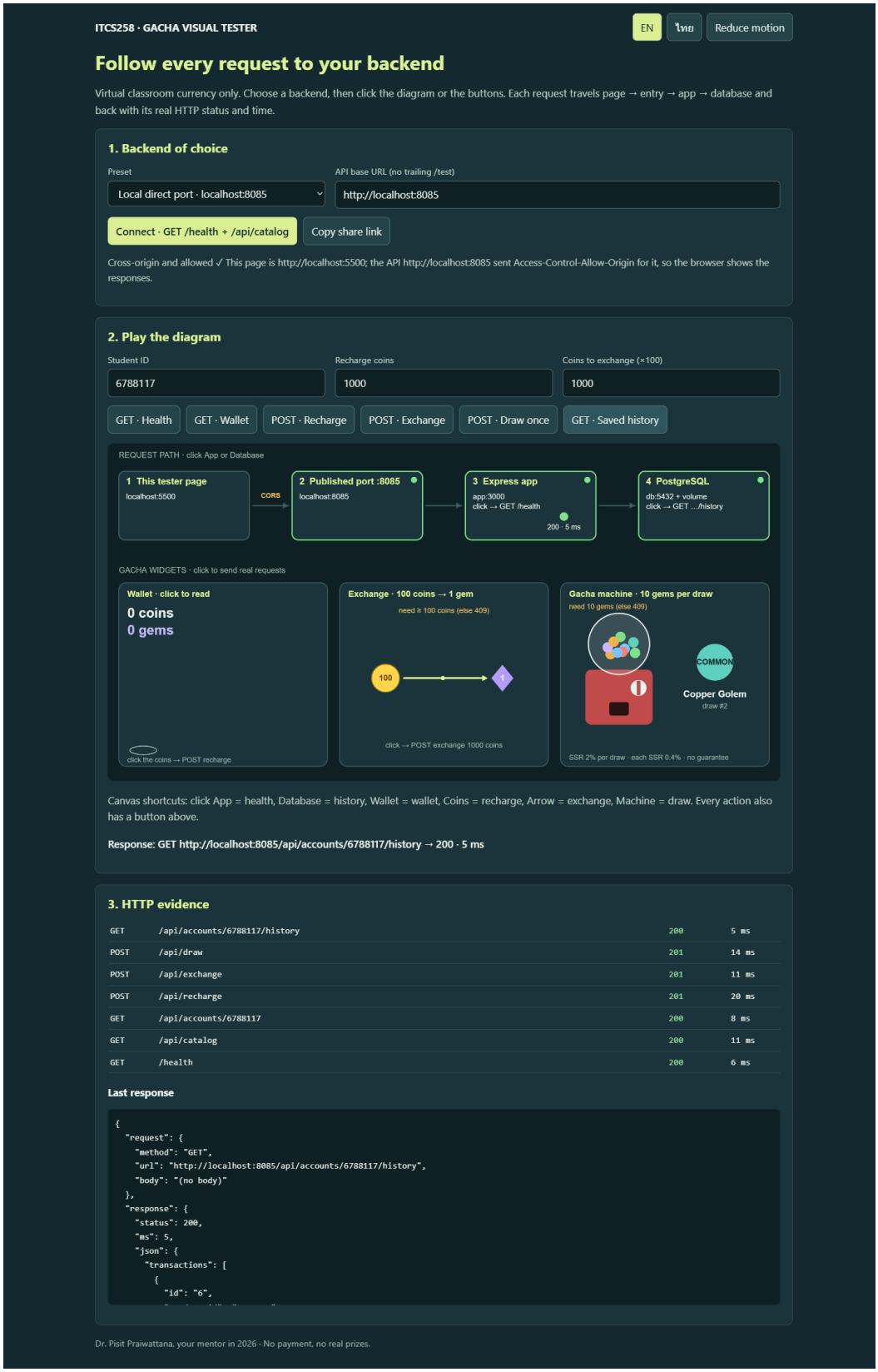
The visual tester shows the whole request path; red means the browser hid the response, green means it read it.



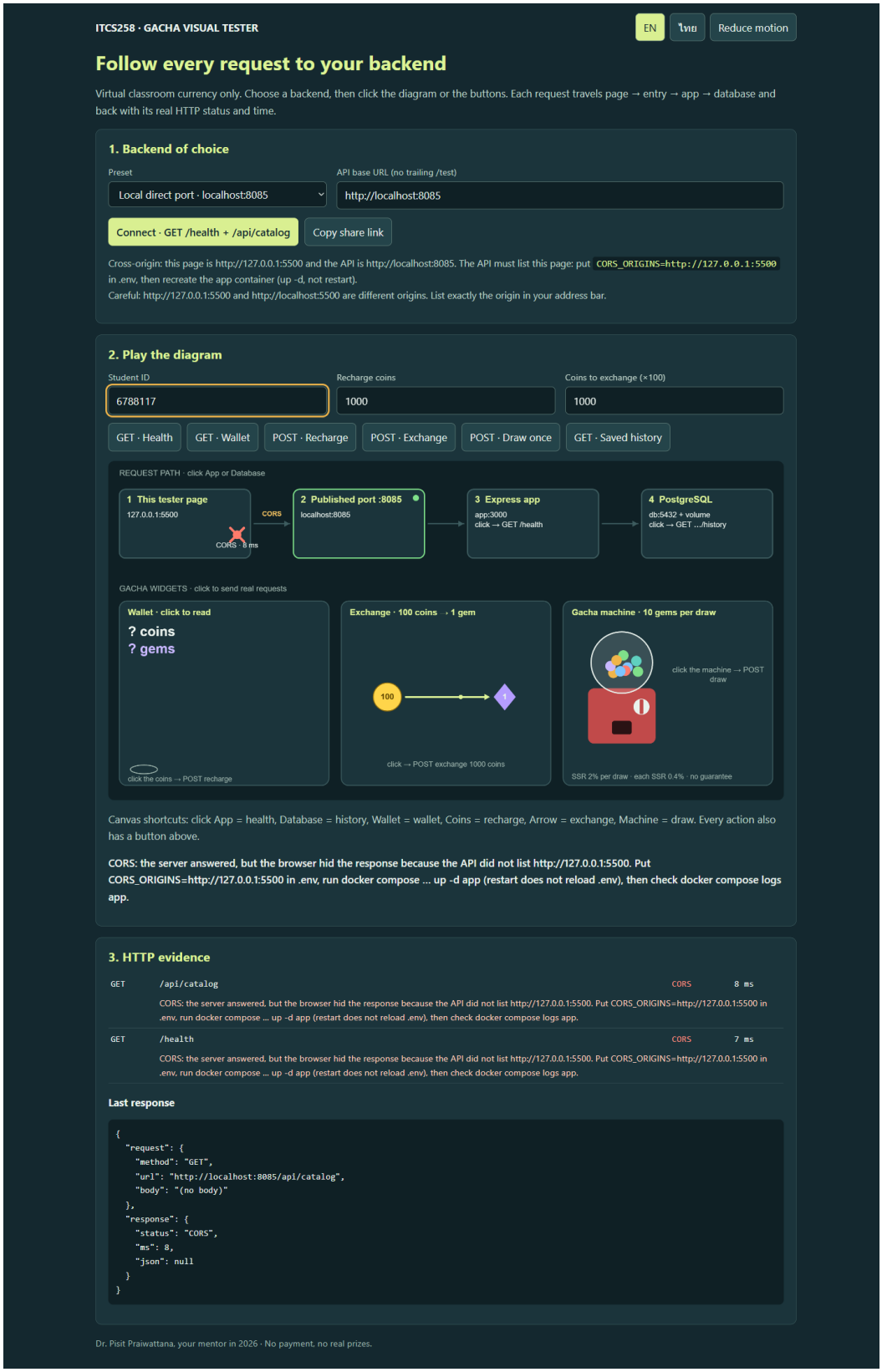
Same origin: http://localhost:8085/visual/ and the API share http://localhost:8085, no CORS needed.



Blocked: page http://localhost:5500, API http://localhost:8085, CORS_ORIGINS empty.



Allowed after recreating: CORS_ORIGINS=http://localhost:5500; recharge/exchange/draw/history all green.



127.0.0.1:5500 is a different origin from localhost:5500, so it is blocked again.

CORS evidence table:

Stage	What I did	Evidence	Result
Same origin	http://localhost:8085/visual/ → Connect	cors-0 screenshot	"Same origin… no CORS setting is needed", /health 200
Blocked	http://localhost:5500 with API http://localhost:8085, CORS_ORIGINS empty	cors-1 + Console	✕ CORS at page node; console says No 'Access-Control-Allow-Origin'

Stage	What I did	Evidence	Result
Server answered	curl -H "Origin: http://localhost:5500" http://localhost:8085/health	terminal	HTTP 200 + health JSON, no Access-Control-Allow-Origin
App log	docker compose … logs app	terminal	CORS_ORIGINS = (empty…); CORS: page origin http://localhost:5500 is not in CORS_ORIGINS (empty)
restart	edit .env, restart app, exec app printenv CORS_ORIGINS	terminal	empty line; curl still has no header
up -d	up -d --wait app, printenv CORS_ORIGINS	terminal	http://localhost:5500; Vary: Origin + Access-Control-Allow-Origin
Allowed	reload http://localhost:5500 → Connect → Recharge → Exchange → Draw → History	cors-2 screenshot	all nodes green, POST 201, GET 200
Preflight	curl -X OPTIONS http://localhost:8085/api/draw …	terminal	204 with ACAO, Allow-Methods, Allow-Headers, Max-Age 600
127.0.0.1 trap	http://127.0.0.1:5500 → Connect	cors-3 + app log	✕ CORS; origin http://127.0.0.1:5500 is not in CORS_ORIGINS (http://localhost:5500)
Through proxy (optional)	http://gacha.test/backend/visual/ → Connect	cors-4 screenshot	Same origin: page and API share http://gacha.test

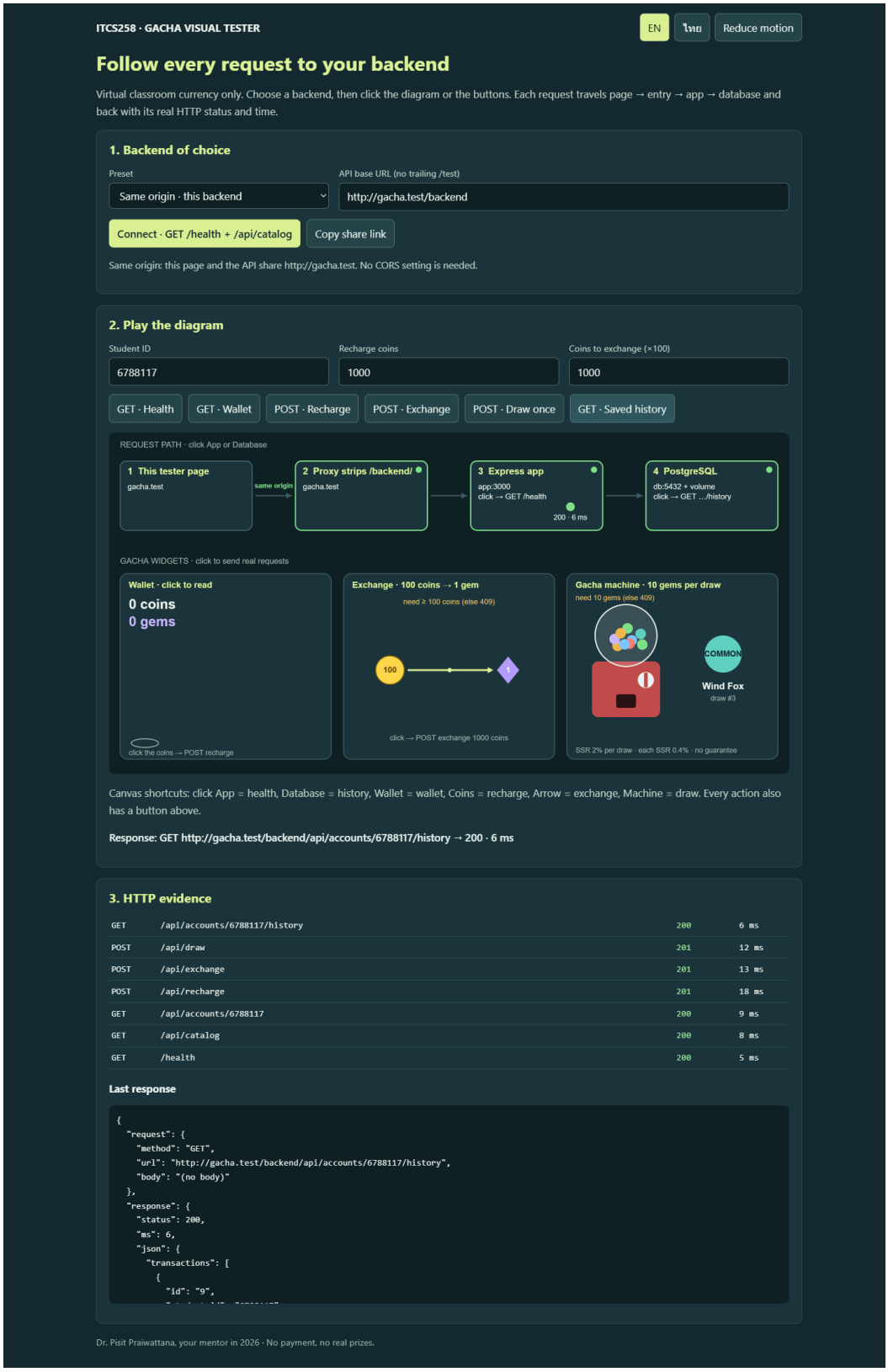
Key outputs:

```
# blocked, server still answered
HTTP/1.1 200 OK
{"status":"ok","database":"connected","app":"gacha","version":"dev"}

# after restart app (new value in .env, old container)
$ docker compose ... exec app printenv CORS_ORIGINS
                     <- empty line

# after up -d --wait app
$ docker compose ... exec app printenv CORS_ORIGINS
http://localhost:5500
Vary: Origin
Access-Control-Allow-Origin: http://localhost:5500

# preflight
HTTP/1.1 204 No Content
Access-Control-Allow-Origin: http://localhost:5500
Access-Control-Allow-Methods: GET, POST, OPTIONS
Access-Control-Allow-Headers: Content-Type
Access-Control-Max-Age: 600
```



Through NPM: the page and the API share http://gacha.test, so no CORS is involved.

Why restart did not apply CORS_ORIGINS: restart only stops and starts the existing container. Its environment was fixed when the container was created, so the new .env value is not read; up -d compares the desired configuration, sees the changed value and recreates the container.

Why 127.0.0.1:5500 and localhost:5500 are different origins: An origin is scheme + host + port. The host names are compared as strings, so 127.0.0.1 and localhost are different origins even though they reach the same server on the same machine.

Why the proxy makes the page and API same-origin (one sentence): NPM serves the page and forwards /backend/ on the same host and port, so the browser sees one origin and never sends a cross-origin request.

Evidence and explanation: The 127.0.0.1 trap is the most useful part: the app log showed CORS_ORIGINS (http://localhost:5500) while the page origin was http://127.0.0.1:5500. Opening the listed origin fixed it without changing the server. CORS_ORIGINS=* would allow every origin, but it is only acceptable for this authless classroom API.

04 — Ubuntu VM as the server / ใช้ Ubuntu VM เป็นเซิร์ฟเวอร์

The second host is an Ubuntu 26.04.1 LTS instance on WSL2 with its own Docker Engine, SSH server and disk. It has the address 172.31.236.39 and uses NAT, not bridged networking: Docker publishes 80/443 on the VM interface and 81 on the VM loopback only, and Windows reaches the VM address directly, so no port forwarding was needed on the laptop. The NPM admin page is opened through an SSH tunnel.

```
Laptop (Windows 11)
  hosts: 172.31.236.39 gacha.test
  browser http://gacha.test/backend/test
        |
        | TCP 80, Host: gacha.test, path /backend/test
        v
Ubuntu VM (WSL2, 172.31.236.39)
  NPM container :80/:443 (admin 127.0.0.1:81, reached with ssh -L 8181:127.0.0.1:81)
        | proxy_pass http://app:3000, path /test
        v
  app container :3000  --->  db container :5432  --->  volume gacha-6788117_pgdata
```

SSH session and basic shell operations on the VM:

```
student@ubuntu-server:~$ pwd
/home/student
student@ubuntu-server:~$ ls -la ~/apps/gacha-workshop | head -5
total 308
drwxr-xr-x 8 student student  4096 Nov 11 11:27 .
drwxr-xr-x 3 student student  4096 Nov 11 11:27 ..
-rwxr-xr-x 1 student student   166 Nov 11 11:27 .dockerignore
-rwxr-xr-x 1 student student   110 Nov 11 11:27 .env
student@ubuntu-server:~$ mkdir -p ~/notes && cd ~/notes && pwd
/home/student/notes
```

VM identification and Docker service:

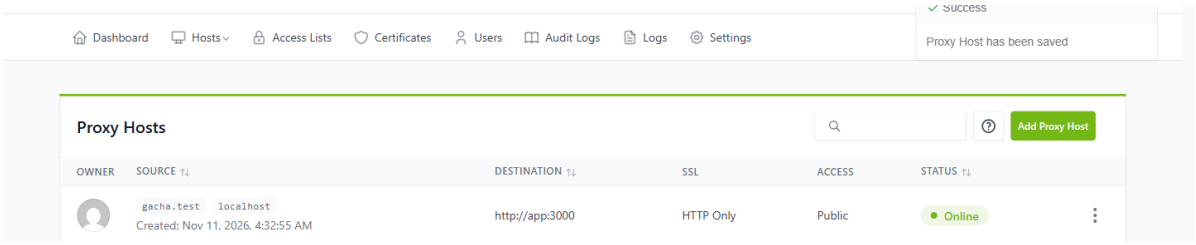
```
student@ubuntu-server:~$ hostnamectl
 Static hostname: ubuntu-server
Operating System: Ubuntu 26.04.1 LTS
          Kernel: Linux 6.18.33.2-microsoft-standard-WSL2
    Architecture: x86-64
student@ubuntu-server:~$ free -h | head -2
               total        used        free      shared  buff/cache   available
Mem:            15Gi       2.5Gi       9.8Gi        61Mi       3.4Gi        12Gi
student@ubuntu-server:~$ df -h / | tail -1
/dev/sdf       1007G  5.5G  951G   1% /
student@ubuntu-server:~$ ip -br address
eth0  UP  172.31.236.39/20
student@ubuntu-server:~$ sudo systemctl status docker --no-pager
     Active: active (running)
student@ubuntu-server:~$ sudo docker compose version
Docker Compose version 2.40.3+ds1-0ubuntu1
```

Copy and startup, from the laptop and then the VM:

```
scp -r gacha-workshop student@172.31.236.39:~/apps/
ssh student@172.31.236.39
cd ~/apps/gacha-workshop
cp .env.example .env && nano .env        # STUDENT_ID=6788117, own .env
sudo docker compose --env-file .env -f compose.yml -f compose.production.yml -f compose.proxy.yml \
  up -d --build --wait
```

NPM on the VM was reached through the tunnel, not by opening port 81:

```
ssh -L 8181:127.0.0.1:81 student@172.31.236.39
# browser: http://localhost:8181  -> new NPM admin for the VM
```

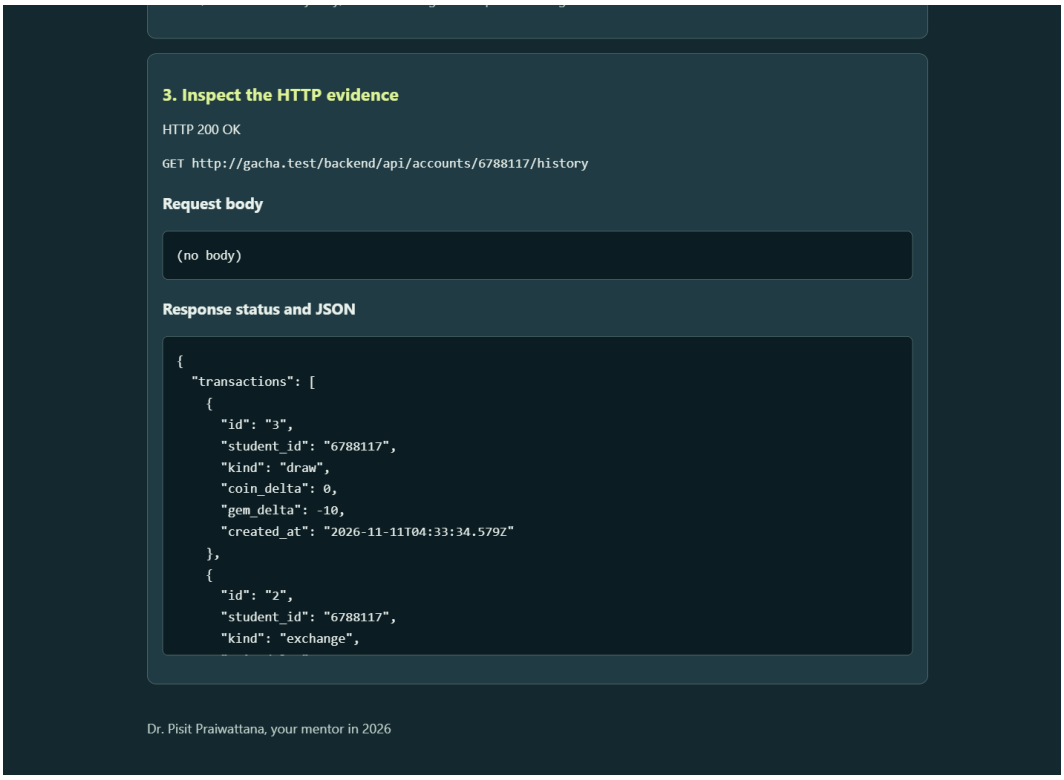


The VM's own NPM: one proxy host for gacha.test + localhost → http://app:3000, Online.

Then the laptop hosts file was changed to the VM address (only one gacha.test line) and the same URL was tested:

```
C:\Windows\System32\drivers\etc\hosts
172.31.236.39 gacha.test

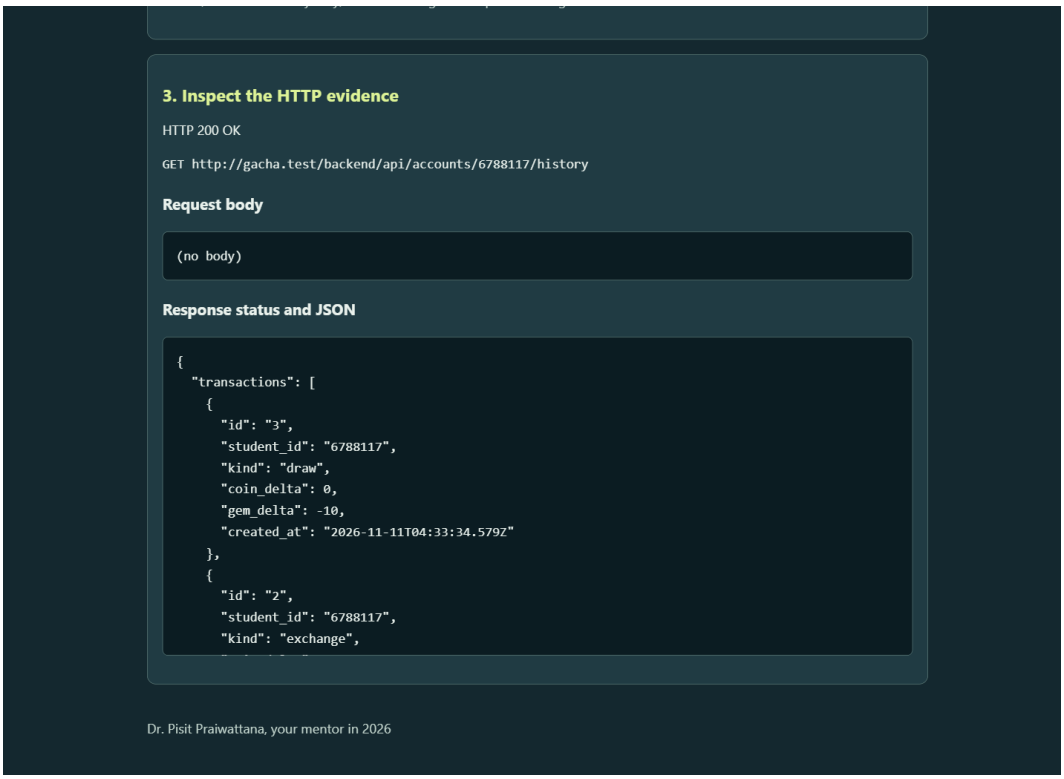
curl.exe -v http://gacha.test/backend/health
*   Trying 172.31.236.39:80...
< HTTP/1.1 200 OK
```



http://gacha.test/backend/test through the VM: Health 200, recharge/exchange/draw 201, history 200.

VM persistence — app container recreated, storage untouched:

Item	Before app replacement	After app replacement
App container ID	9219ea89b0af…	b9e6ed93402c…
DB volume identity	gacha-6788117_pgdata	gacha-6788117_pgdata
Original VM draw ID + created_at	1 / 2026-11-11T04:33:34.579Z	1 / 2026-11-11T04:33:34.579Z



Saved history after up -d --force-recreate --wait app: same draw id 1, Stone Knight, same time.

```
sudo docker compose ... ps -q app
9219ea89b0afe5f51199d7f70fb28146f14100b23b456fc4750d56262d084f4e
sudo docker compose ... up -d --force-recreate --wait app
sudo docker compose ... ps -q app
b9e6ed93402c62e52f8af2a8cbe7c79255a120a1eac764b341e4640a6406c3c6
sudo docker volume ls
local  gacha-6788117_pgdata
```

Separate storage: The VM has its own /var/lib/docker and its own gacha-6788117_pgdata volume. Draw IDs start again at 1 and the laptop draws are not present, so the storage really is separate. Recreating the app container kept the same database and the same draw.

Public VPS requirements: A public VPS needs a real domain with a DNS A/AAAA record to the server, provider firewall rules, and only ports 80/443 plus the SSH port exposed. NPM's SSL tab requests the Let's Encrypt certificate and Force SSL, which is impossible with the reserved .test name. Allow SSH before enabling UFW so the session is not lost; Docker-published ports can bypass UFW, so app, DB and admin ports stay unpublished or loopback-only. The developer still owns updates, backups, log review, certificate renewal and access control.

Diagnosis and reflection / วิเคราะห์ปัญหาและสะท้อนผล

Symptom → hypothesis → check → fix

After I first saved the localhost proxy host, /health returned 502 while the Proxy Hosts list still said Online. My hypothesis was a wrong upstream name, not a broken app. I checked the NPM error log and read "localhost could not be resolved (3: Host not found)", then confirmed the app itself was healthy through the /backend/ location. I changed Forward Hostname / IP back to app and /health returned 200. The preventive check is to open /health after every NPM save and to remember that Online only means the nginx configuration loaded.

Why production publishes no ports, and what the proxy does instead: Production publishes no app or database ports because only NPM is exposed to the outside world. NPM terminates HTTP/HTTPS, chooses the proxy host by Host header and the location by path, rewrites /backend/ and forwards to app:3000 over the Compose network. The database stays on the private network with no host port.

How NPM chose the proxy host and location for http://gacha.test/backend/test: NPM compared the Host header gacha.test with the server_name list of each proxy host and chose the host that contained both localhost and gacha.test. Inside that server it used the longest matching prefix, /backend/, applied the rewrite and proxied the rest of the path to app:3000.

What changed between laptop and VM: The URL, DNS/hosts entry, TLS and storage change between the laptop and the VM. What does not change is the developer's work: keeping images and base images updated, monitoring logs and disk, backing up the pgdata volume, renewing certificates, limiting public ports and reviewing the proxy configuration before production.

Self-quiz topic to revisit: Why restart does not reload .env, how Compose decides to recreate a container, and longest-prefix matching for /backend/. I expected restart to apply the new origin, so I will review that pitfall before the quiz.

In-room check and final submission / ตรวจในห้องและส่งงาน

[X] I completed 01–04 using the same working example.

[X] I demonstrated my actual experiment to the LA in the room.

[X] All evidence is from my own screen and readable.

[X] My report includes the merge evidence, my four NPM screenshots, the URL test tables, the CORS evidence table, request paths, persistence comparisons and repository URL.

[X] No actual passwords or tokens are visible.

[X] The single PDF is named student_ID_lab13.PDF, using my actual ID (6788117_lab13.PDF).

[X] I uploaded it to MyCourse by its deadline and opened the uploaded PDF to verify it.